

esade

Prototyping with Dash

Jose A. Rodríguez-Serrano



Dash is a Python package for building web applications.

- Developed by Plotly.
- Written on top of Flask and React.js.
- Like Streamlit, Dash allows building web applications by specifying web components, and their interactions, in python.
- Specially useful for data apps and dashboards, since it integrates well with Plotly.

Differences with Streamlit

Advantages:

- ✓ In Dash, you work directly with the tree of the HTML document, which gives more freedom for styling.
- ✓ Dash works well with Plotly, a very powerful python plotting library. You can "react" to e.g. clicks in plots (not possible in Streamlit).

Disadvantages:

- ✗ In Dash, knowing some web design concepts, such as HTML components and CSS style sheet, is helpful.
- ✗ In Dash, you need a bit of code to specify interactions (callbacks).

01

First Dash Example



First Dash example: Structure

app.py

```
from dash import Dash, html, dash_table, dcc

...

app = Dash(__name__)

app.layout = ...

if __name__ == '__main__':
    app.run(debug=True)
```

First Dash example: Layout

app.py

```
app.layout = html.Div([
    html.H1("Real estate prices in France"),
    html.P(children=[
        "Dash app to display real estate prices in Paris. ",
        "The data is available on the French government's ",
        dcc.Link("Open Data Portal", href="https://your-source")
    ]),
    html.H2("Raw data"),
    dash_table.DataTable(
        id='table',
        columns=[{"name": i, "id": i} for i in df.columns],
        data=df.to_dict('records'),
    )
])
```

Understanding Dash

Dash provides python objects for:

- Dash core components (`dash.dcc`). Similar to the Streamlit widgets.
- HTML components, such as H1, H2, Div, Paragraph... (`dash.html`).
- Specific advanced components such as the DataTable (e.g. `dash.dash_table`).

The layout tree

- The first argument of many Dash components is a list of **children** components.
- Children can be a single value or a list of values. "Values" can be other Dash components, strings or numbers.
- The `app.layout` points to the **root component**, usually a container component, such as `html.Div`.
- The root component and all the recursive children make the **layout tree**.

(This is closely related to the DOM in HTML jargon).

02

Adding interactivity



Interactivity in Dash

app2.py

```
app.layout = html.Div([
    ...
    dcc.Dropdown( ← We add a dropdown component
        id='year-dd',
        options=['2019', '2020', '2021', '2022'],
        value='2022'
    ),
    html.Div(id='table-container') ← We remove the table and leave a placeholder
])
```

💡 Also note we assign an id to each component.

The callback

A callback function is marked with the decorator `@callback`.

- It is a special function that is called automatically when certain conditions occur.
- In a basic callback you need to specify a) an output , b) an input and c) a function.

Example callback (app2.py)

```
@callback(  
    Output('table-container', 'children'),  
    Input('year-dd', 'value') x  
)  
def update_output(value):  
    fname = get_file(value)  
    df = prepare_data(fname)  
    return dash_table.DataTable(  
        id='table',  
        columns=[{"name": i, "id": i} for i in df.columns],  
        data=df.to_dict('records'),  
    )
```

The way to read this is:

- When the attribute `value` of the component with id `year-dd` changes,
- the attribute `children` of the component with id `table-container` will be replaced
- by the `return` value of the function.

The callback

Output

- The first argument is the **id** of the component in the layout that will be affected by the change.
- The second argument is the "attribute" affected by the change (usually "children").

Input

- The first argument is the **id** of the component that **triggers** the change.
- The second argument is the "attribute" whose state changes. Each Dash component defines "attributes" subject to triggering a callback when changing.

The callback

Callback function

- It must return an object compatible with the attribute specified in output (e.g. in case of "children", a string, number, Dash component, or list thereof).
- The name of the function does not matter.
- The argument to the function is used to track the new value of the input.
- The name of the argument does not matter.

Exercise: Add a map

- Open file `app2b.py`
- You have a function called `get_map` imported (from `common.py`).
- The layout includes a new component `dcc.Graph` (this is the placeholder for the map).

Instructions

Write a callback to display the map when the selected year changes.

Hints:

- Use the code template provided in the file.
- Make use of `get_map`.
- The attribute of the `dcc.Graph` object that needs to be changed to redraw the map is called `figure`.

03

A word on layout



CSS Stylesheets

CSS (Cascading Style Sheets) is a language to provide the look and feel of HTML websites

In Dash, you can add CSS stylesheets to format your components.

Using a static CSS file

- Place CSS file(s) under a folder **assets/**
- Give a class name to components using the argument **className**.
Example: `dcc.Dropdown(id='year-dd', className="dd1", ...`

Using an external CSS URL

- When initializing the Dash app, add the argument **external_stylesheets=**
- Example: `app = Dash(__name__, external_stylesheets=[...`

Dash Bootstrap Components

Specifying rows and columns

```
import dash_bootstrap_components as dbc
...
dbc.Container([
    dbc.Row([
        dbc.Col([dcc.Graph(id='map')], width=9),
        dbc.Col([html.Div(id='info')], width=3)
    ])
])
```


Dash Bootstrap Components

- `dbc.Row` and `dbc.Col` behave as regular Dash components (can be and can have children).
- They always need to be inside a `dbc.Container`.
- We can specify the `width` of each column (in integer units out of a total of 12).
- For all this to work, you need to add the following argument when init'ing the app:

```
app = Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])
```

Dash bootstrap components package

<https://dash-bootstrap-components.opensource.faculty.ai/>

04

Adding and interacting with plots

A teal-colored decorative shape, resembling a stylized hill or a triangle, is positioned at the bottom of the slide, partially overlapping the text.

Plotly in Dash

- In Dash, the component **dcc.Graph** allows adding plots produced with the Plotly package.
- Plotly allows creating interactive plots. Therefore, it is especially interesting for dashboards, web applications and prototypes.
- In Plotly the whole plot metadata is stored as a Python dictionary which is highly customizable and easy to modify.
- In Dash we can define callbacks for interactions with **dcc.Graph** objects, such as clicking on a point or hovering the mouse.

(The latter is not possible in Streamlit or with the conventional Matplotlib package).

dcc.Graph

<https://dash.plotly.com/dash-core-components/graph>

Callbacks on graphs

From the Dash documentation

The `dcc.Graph` component has four attributes that can change through user-interaction: `hoverData`, `clickData`, `selectedData`, `relayData`.

Dash docs: Interactive graphing

<https://dash.plotly.com/interactive-graphing>

Callbacks on graphs

Hence, to capture info about the click point, we can define a callback whose input specifies the graph component (here, with ID 'map'), and the triggering attribute ('clickData'):

app3.py

```
@callback(
    Output('info', 'children'),
    Input('map', 'clickData')
)
def display_click_data(click_data):
    """Update the information with the selected property."""
    return format_property_data(click_data)
```

Sharing data between callbacks

- Here we used the trick of passing all the necessary information of a property to the **customdata** argument of the plot.
- In the callback, the **clickdata** attribute has access to the **customdata**.
- In more advanced use cases, you might want to share the whole dataframe.
- Don't use global variables! (not recommended in general + they do not work well in Dash)

From the Docs: Sharing data between callbacks

```
https://dash.plotly.com/sharing-data-between-callbacks
```

More links

Dash Gallery

`https://dash.gallery/`

Dash core components

`https://dash.plotly.com/dash-core-components`

Plotly

`https://plotly.com/python/`

INTERACTIVE COURSE

Building Dashboards with Dash and Plotly

[Start Course](#) [Bookmark](#)

●● Intermediate ⌚ 4 hours ▶ 15 videos <> 47 exercises 👤 10,849 participants 3900 XP

Datacamp course

<https://app.datacamp.com/learn/courses/building-dashboards-with-dash-and-plotly>

esade

Questions?

Contact: joseantonio.rodriguez15@esade.edu
Office direction: SC-207E (Campus Sant Cugat)